TECHNICAL GUIDE

Momentum Internals

One React codebase → macOS desktop app, installable PWA, and web app — all sharing a single cloud SQLite database on Turso. Zero hosting cost.

Live App | GitHub Repo

Front-end · Back-end · Database

LAYER	WHAT IT IS
Front-end	**React 18 + Vite + Tailwind CSS.** Everything you see — every screen, chart, and interaction — is one React single-page app. The same built bundle runs in three shells: the Electron desktop window, the installed Android PWA, and any web browser.
Back-end	**There is no traditional backend server — by design.** In cloud mode the front-end talks *directly* to the database over HTTPS (a "serverless" architecture): nothing to host, nothing to keep awake, nothing to pay for. All business logic (queries, aggregations, reports) lives in the data layer itself. Two situational stand-ins exist: on desktop-local mode, the **Electron main process** acts as a mini-backend (IPC → SQLite); in LAN mode, a small **Node HTTP server on the Mac** (port 3001) plays the role.
Database	**SQLite — in two homes, same schema (19 tables).** Primary: **Turso**, a cloud-hosted SQLite (libsql) in Mumbai that all devices share over HTTPS. Secondary: a plain **SQLite file on the Mac** (via better-sqlite3), used in desktop-local mode and kept as a frozen offline backup.

Architecture

Every UI component talks to a single abstraction — `window.api` (~75 async functions). At startup, `src/utils/bootstrapApi.js` picks which backend implements it, in priority order:

turso 1

Cloud

Creds in localStorage → SQL over HTTPS to Turso. Shared across all devices.

electron 2

Desktop local

IPC → main process → better-sqlite3. Used when no cloud creds saved.

rest 3

Mac server

Browser served from the Mac (WiFi/Tailscale) → REST on :3001.

setup 4

First run

Nothing available → CloudSetup connect screen.

Why window.electronApi, not window.api, in the preload

`contextBridge.exposeInMainWorld` creates *read-only* properties. `window.api` must stay assignable so cloud mode can take over on desktop — the bootstrap maps `window.api = window.electronApi` when local mode wins.

Stack

LAYER	TECH
UI	React 18 · Vite · Tailwind (custom `th-*` tokens, dark/light) · Recharts · @dnd-kit · lucide-react
Desktop	Electron · better-sqlite3 · built-in HTTP API server (`electron/api-server.js`) for WiFi/Tailscale PWA
Cloud DB	Turso (libsql — SQLite over HTTPS) · region `aws-ap-south-1` Mumbai
Hosting	GitHub Pages · `gh-pages` branch · $0

Project structure

```
electron/
  main.js            Electron main process + all IPC handlers
  preload.js         contextBridge → window.electronApi
  database.js        ★ SOURCE OF TRUTH for all SQL (better-sqlite3, sync)
  api-server.js      HTTP mirror of the API for the mobile/Tailscale PWA
src/
  data/
    tursoDb.js       ★ AUTO-GENERATED async port of database.js — do not hand-edit
    queryClient.js   libsql bridge: q.all/get/run/exec · creds in localStorage
    tursoApi.js      maps the window.api surface → tursoDb functions
  utils/
    bootstrapApi.js  backend selection (see Architecture)
    apiPolyfill.js   REST-backed window.api for Mac-server mode
  components/        feature folders (Dashboard, Projects, Calendar, …)
scripts/
  port-db-to-turso.mjs   regenerates tursoDb.js from database.js
  patch-turso-port.mjs   post-transform fixes — run after the port script
  migrate-to-turso.mjs   local→cloud data copy (WIPE=1 = fresh snapshot)
  deploy-pages.sh        build + publish to GitHub Pages
```

The generated data layer

Golden rule

`src/data/tursoDb.js` is **generated, never hand-edited**. All SQL lives once, in `electron/database.js`.

After changing any query there:

```
node scripts/port-db-to-turso.mjs && node scripts/patch-turso-port.mjs
npx vite build   # surfaces any await-in-non-async misses
```

The transform converts sync better-sqlite3 calls to `await q.*`. Hand-patched cases live in the patch script: the one transaction (`setDailyIntentions`), reused prepared statements, named/positional arg mixes, and `.map()` callbacks containing awaits. Adding new code with those patterns? Extend the patch script.

Development, builds & deploys

```
npm install
npm run dev          # Electron + Vite dev (desktop)
npx vite             # web-only dev (boots to CloudSetup without creds)
```

TARGET	COMMAND	RESULT
Web (Pages)	`bash scripts/deploy-pages.sh`	live in ~1 min
Desktop DMG	`npm run build`	`release/Momentum-1.0.0-arm64.dmg`

Stuck Pages deploy?

Builds occasionally hang GitHub-side. Check & force:

```
gh api repos/Zubshamid786/momentum/pages/builds/latest --jq .status
gh api repos/Zubshamid786/momentum/pages/builds -X POST   # force rebuild
```

No CI: the Actions workflow in `.github/` is gitignored (the `gh` token lacks `workflow` scope). To enable push-to-deploy: `gh auth refresh -s workflow`, then un-ignore it.

Cloud data & credentials

- Turso URL + token are entered once per device, stored **only in that device's localStorage** — never in the (public) repo.
- Schema is created idempotently on first connect (`initSchema()`), gated by a localStorage flag.
- Fresh cloud snapshot from the Mac's local DB:

```
WIPE=1 TURSO_DATABASE_URL=libsql://… TURSO_AUTH_TOKEN=… node scripts/migrate-to-tur
```

  Uses Node's built-in `node:sqlite` — better-sqlite3's binary is often left x64 by Electron builds.

- Token rotation: Turso dashboard → revoke → generate → re-enter per device (Settings → Cloud Sync).

Hard-won gotchas

1. **Tailwind `md:` breakpoints are unreliable in the Android PWA.** Use a ResizeObserver on the container (see `MountainView.jsx`) or explicit props (`TaskDetail`'s `panel` prop).

2. **SQLite `date('now')` is UTC.** Date-only timestamps must be anchored at *noon local* so they land on the right calendar day in every timezone.

3. **PWA on a subpath needs relative manifest paths** — `start_url: "./"`, `scope: "./"`, and real 192/512 PNG icons — or Android installs a Chrome-badged shortcut instead of a WebAPK.

4. **`(await …)` statement lines need a leading `;`** in generated code — ASI can glue them to the previous line as a call expression.

5. **libsql rejects `undefined` args** (better-sqlite3 tolerated extra named params) — `queryClient.cleanArgs` strips them.
